

Highlights

What Drives Small Object Detection Accuracy in UAV Imagery? A Controlled Study of Training Resolution, Detection Stride, and Feature Modules

Moatassim Barakat, Xinzhong Zhu

- Single-resolution training beat a four-phase resolution curriculum by 3.9 pp
- It matched the curriculum's final accuracy using a third of the compute
- Edge selection and CBAM-style recalibration gave no significant gain
- Neither multi-scale dilation nor adaptive gating beat a plain convolution
- Input scale and stride set how many anchor points fall on a small object

What Drives Small Object Detection Accuracy in UAV Imagery?

A Controlled Study of Training Resolution, Detection Stride, and Feature Modules

Moatassim Barakat¹, Xinzhong Zhu¹,

*^aCollege of Computer Science and Technology, Zhejiang Normal University, 688 Yingbin
Road, Jinhua, 321004, China*

Abstract

Small-object detectors for drone imagery are often built by attaching modules to a YOLO baseline, with gains of one to three points usually measured in a single training run. We asked which design decisions still matter once the training budget, evaluation settings and random seed are controlled. Using YOLOv11s on VisDrone-DET2019, we compared a four-phase progressive resolution schedule with single-resolution training at the same epoch budget, added a stride-4 detection level, and tested multi-scale edge information selection (MEIS), cross-scale weighted fusion and CBAM-style recalibration. Differences were tested with paired image-level bootstrap and permutation tests. Training at 1280 px in a single phase outperformed the progressive schedule by 3.9 points mAP@50 at 1280 px (57.3% vs. 53.3%, $p < 0.001$) and matched the schedule’s final accuracy after one third of its compute. MEIS with recalibration changed mAP@50 by +0.6 and −0.3 points on two seeds, neither significant, and a reduced-budget factorial ablation found no benefit from multi-scale dilation or input-adaptive gating over a plain convolution of the same size. The stride-4 level changed mAP@50 by **[X]** points. We explain these results with a sampling argument: input scale and feature stride fix how many anchor points fall on a small object, and modules that act after downsampling cannot add them back. The best configuration reaches 58.8% mAP@50 at 1536 px with 9.9M parameters. On UAVDT, which contains only vehicles, **[UAVDT result]**; generalization to small non-vehicle objects

*Corresponding author.

outside VisDrone is untested.

Keywords: Small object detection, UAV imagery, Sampling resolution, Training schedule, Ablation study, Statistical significance, YOLOv11

1. Introduction

Drones photograph scenes from tens to hundreds of metres above the ground, so the objects that matter for traffic monitoring, crowd counting or search and rescue often cover only a few dozen pixels [? ?]. In the VisDrone-DET2019 training set, 63% of the annotated instances have a geometric-mean side shorter than 32 pixels in the original frame. After the frame is resized for the network, such an object falls on a handful of cells of a stride-8 feature map, and a detector that sees only a few samples of an object has little to work with, however its later layers are designed.

Much of the recent work on this problem modifies the network. Attention blocks, edge enhancement branches, weighted feature fusion and extra prediction heads have all been attached to YOLO baselines [? ? ?], and the reported improvements are usually modest; MEIS-YOLO, for example, reports 1.4 points of AP@50 over its YOLOv11 baseline [?]. Two things make gains of that size hard to interpret. If the proposed model and its baseline are not trained under the same conditions, a gain that comes from a longer schedule or a larger input can end up credited to the new module. And networks trained from different random seeds reach different accuracies [?]; for detectors on VisDrone we measured seed-to-seed differences of up to 0.7 points, which is comparable to the effects being claimed.

This paper reports what happened when we removed those confounds. We started from YOLOv11s [?] and three modules that target the usual explanations for poor small-object accuracy: a multi-scale edge information selection (MEIS) block built from dilated depthwise-separable branches with input-adaptive gating, following the idea of MEIS-YOLO [?]; a cross-scale weighted fusion neck in the spirit of CF-YOLO [?] and BiFPN [?]; and a CBAM-style recalibration module [?] placed after fusion. We also studied two changes that act on sampling rather than on features: the input resolution used for training, reached either directly or through a four-phase progressive schedule, and a stride-4 (P2) detection level. Every comparison uses a matched epoch budget and one evaluation protocol, and each was repeated with independent seeds and tested with paired resampling over the

validation images.

The results did not follow the module-centred story we expected. Single-resolution training at 1280 px was 3.9 points better than the progressive schedule at the same epoch budget, and it reached the schedule’s final accuracy with one third of its compute. MEIS with recalibration produced no significant change on either of two seeds. A factorial ablation showed that neither of the two ingredients that define MEIS, multi-scale dilation and input-adaptive gating, did better than a plain convolution with the same number of parameters. The stride-4 level **[summarise stride result]**. This pattern is what one would expect if accuracy on small objects is limited mainly by how densely each object is sampled, and Section ?? makes that argument concrete for anchor-free detectors.

The paper makes three contributions: (i) a matched-budget comparison of progressive and single-resolution training for small-object detection, with measured compute and paired significance tests; (ii) a controlled evaluation of edge selection, weighted fusion and recalibration modules, including a factorial ablation that separates multi-scale dilation, adaptive gating and added capacity; and (iii) a sampling account of these results that gives, for a given input size and stride, the smallest object that can receive positive training samples.

2. Related Work

2.1. Feature modules for small aerial objects

Feature pyramids [? ?] and weighted bidirectional fusion [?] are the standard way to give a detector features at several scales, and channel or spatial attention [? ? ?] is often added to suppress the cluttered backgrounds of aerial scenes. Detectors designed for drone imagery combine these ideas. TPH-YOLOv5 [?] adds transformer prediction heads, CBAM and an extra prediction head for tiny objects. MEIS-YOLO [?] inserts an edge information selection block, an asymptotic feature pyramid and bi-level routing attention into YOLOv11. CF-YOLO [?] replaces the YOLOv5s neck with cross-fusion blocks for detection in snow. These papers report their gains without estimates of run-to-run variation, which makes differences of a point or two difficult to judge. When a design adds a higher-resolution head together with new blocks, as TPH-YOLOv5 does, the change in sampling alone may account for part of the gain.

2.2. Resolution and sampling in aerial detection

A second line of work treats small objects as a resolution problem. Coarse-to-fine detectors such as ClusDet [?], DMNet [?] and UFPMP-Det [?] locate object-dense regions and process them at higher magnification. Query-Det [?] runs a cheap coarse pass and computes high-resolution features only where small objects are likely, and SAHI [?] slices large images into overlapping tiles. These methods show that magnification helps, but they also change the inference pipeline and its cost. We look at the two simplest sampling controls available inside a single-pass detector: the input resolution used in training and the stride of the finest detection level.

2.3. Resolution schedules during training

Curriculum learning [?] orders training from easy to hard, and progressive resizing applies a similar idea to image resolution. EfficientNetV2 [?] trains image classifiers on small images first and raises resolution and regularization together, mainly to shorten training. Touvron et al. [?] showed that test-time resolution interacts with training resolution, so a classifier can benefit from being tested at a larger size than it was trained on. Both lines of work concern classification. For dense detection of tiny objects, where a low training resolution can shrink objects below the size a detection level can represent, we are not aware of a matched-budget comparison between progressive and single-resolution training.

2.4. Variance and significance in detector evaluation

Networks trained with different seeds, data orders and initializations reach different accuracies, and Bouthillier et al. [?] showed that ignoring this variation leads to unreliable conclusions in machine learning benchmarks. When a metric is computed over a test set, paired resampling tests [? ?] offer a practical check, because both models are scored on the same resampled images. Detection papers rarely report them. We apply them at the image level and report separately how much results move between seeds.

3. A Sampling View of Small Object Detection

A detector does not see an object at its original size. A frame whose longer side is W_0 pixels is resized so that this side equals the network input size S , which scales an object of side a to ka with $k = S/W_0$. A feature

level with stride r then samples the resized image once every r pixels, so the number of samples across the object is

$$n = \frac{k a}{r} = \frac{S a}{r W_0}. \quad (1)$$

Seen as a sampling problem, n is the sampling rate relative to the object. With fewer than about two samples per side, a level cannot represent the object’s spatial structure, and the layers that follow inherit the loss [? ?].

For anchor-free YOLO detectors the consequence is more direct. YOLOv11 assigns training targets with the task-aligned assigner of TOOD [?], which only considers anchor points whose centres lie inside a ground-truth box. At a given level an object covers roughly $n_w n_h$ anchor points, where n_w and n_h count samples along its width and height. When both counts are below one, the object may contain no anchor point at that level. If that happens at every level, the object contributes no positive sample and is learned only as background. Setting n to one or two in Eq. (??) gives the smallest object side that can be assigned a target, and the smallest that is sampled twice per side:

$$a_{\min} = \frac{n r W_0}{S}. \quad (2)$$

Fig. ?? illustrates the effect and Table ?? evaluates Eq. (??) for frames 2000 pixels wide, which is about the size of the largest VisDrone-DET2019 frames. Halving the stride and doubling the input size have the same effect on $a_{\min}$. At an input size of 640 px a stride-8 level cannot assign targets to objects smaller than 25 pixels, a range that contains [X]% of the VisDrone training instances; at 1280 px with a stride-4 level the threshold drops to 6.3 pixels ([Y]% of instances below it).

The argument leads to three predictions that the experiments can check.

- P1. Training at a larger input size, or adding a finer detection level, should help most on small objects, because both raise n for the same frames.
- P2. A module that operates on stride- r features cannot raise n . It can reweight what was sampled, but it cannot give training signal to objects that received no positive samples.
- P3. A progressive schedule that starts at low resolution spends its early epochs in a regime where many small objects cannot be assigned targets (Table ??, 640 and 800 px). Under a fixed epoch budget it should therefore do worse than training at the final resolution throughout.

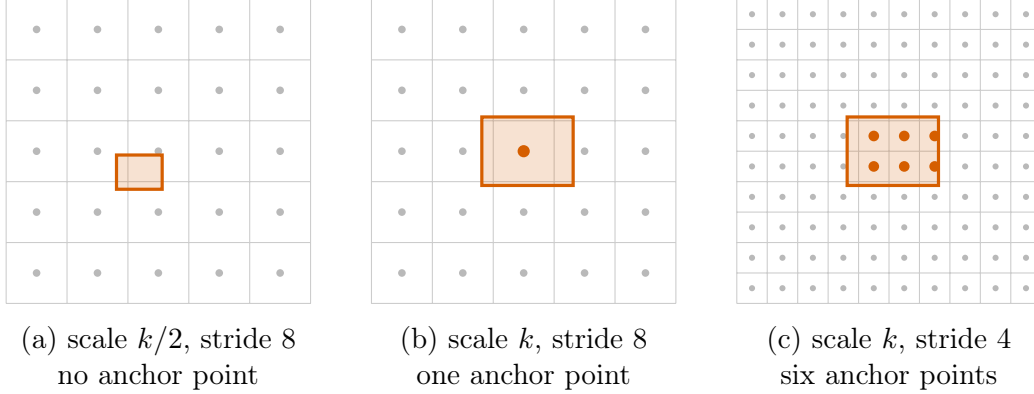


Figure 1: The same object (red box) under three sampling settings. Grid cells span one stride in network-input pixels and dots mark anchor points; filled red dots lie inside the object and can be assigned as positive samples. Halving the input scale (a) removes the only anchor point that the object covers at stride 8 (b), while a stride-4 level (c) places six anchor points on it.

Table 1: Smallest object side $a_{\min}$, in pixels of a 2000-pixel-wide frame, that yields at least n anchor points per side (Eq. ??).

Stride r	n	Network input size S (px)				
		640	800	1024	1280	1536
4 (P2)	1	12.5	10.0	7.8	6.3	5.2
	2	25.0	20.0	15.6	12.5	10.4
8 (P3)	1	25.0	20.0	15.6	12.5	10.4
	2	50.0	40.0	31.3	25.0	20.8

Eq. (??) is a simplification. Mosaic and scale augmentation change k from one training image to the next, context around an object can support detection, and the assigner’s behaviour near box borders depends on implementation details. We use the argument to organise the experiments, not as a quantitative model of accuracy.

4. Experimental Setup

4.1. Dataset

We use VisDrone-DET2019 [?], which provides 6,471 training images and 548 validation images annotated with ten object categories. All results

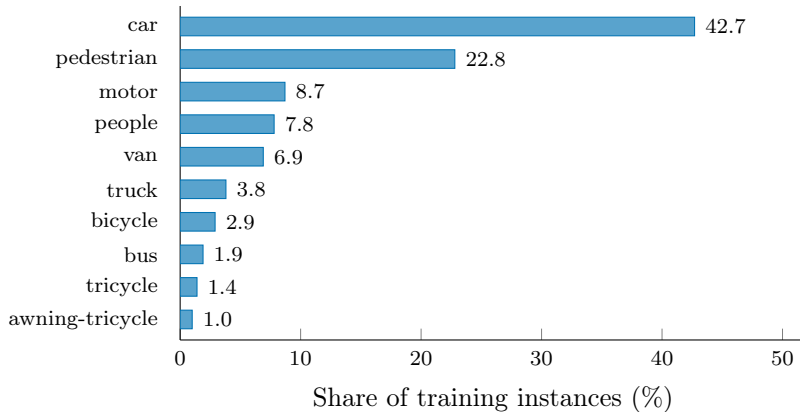


Figure 2: Class distribution of the 305,974 training instances in VisDrone-DET2019 after conversion.

in this paper are on the validation set; we did not use the test-dev split. Annotations were converted to the YOLO format, keeping categories 1 to 10 and discarding ignored regions (category 0) and the *others* class (category 11). Boxes were clipped to the image, and boxes one pixel wide or high or smaller after clipping were removed. The converted training set contains 305,974 instances and the validation set 38,759. The class distribution is strongly skewed (Fig. ??): cars and pedestrians account for 65.5% of training instances, while tricycles, awning-tricycles and buses together make up 4.3%.

4.2. Detectors and modules

All detectors are built on YOLOv11s [?] and initialized from its COCO-pretrained weights. The stock model detects at strides 8, 16 and 32 (P3 to P5). The *Baseline-P2* variant extends the top-down path of the neck to the stride-4 backbone features and starts the bottom-up path from there, which gives four detection levels (Fig. ??). Where a modified neck contains C3k2 blocks whose weight shapes match the original head, those weights are copied as well, so that no variant starts from a randomly initialized block when a pretrained counterpart exists.

Multi-scale edge information selection (MEIS). MEIS is applied to each backbone output that feeds the neck. For an input $X \in \mathbb{R}^{C \times H \times W}$, three depthwise-separable branches with dilation $d \in \{1, 2, 4\}$ produce edge responses E_d , and

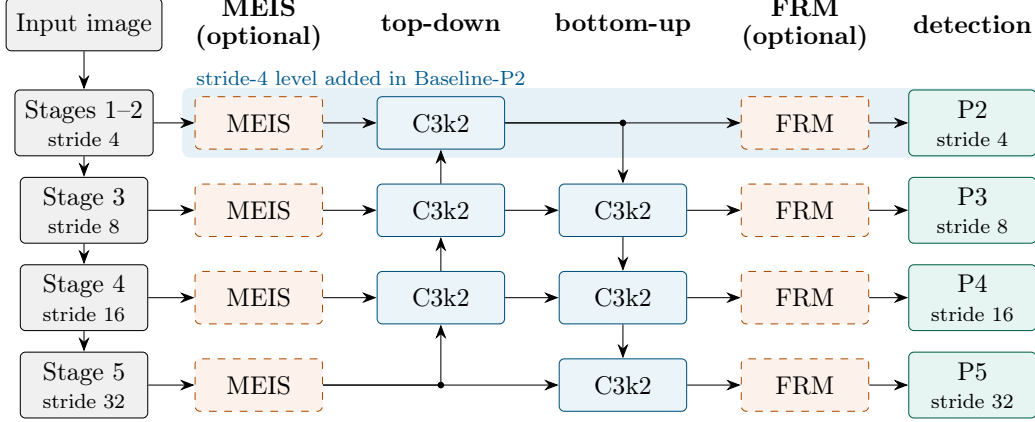


Figure 3: Detector variants. The Baseline-P2 neck extends the YOLOv11s path aggregation neck to the stride-4 backbone features (shaded row). Each C3k2 node in the top-down path follows an upsampling and concatenation, and each node in the bottom-up path follows a strided convolution and concatenation. Dashed blocks are the optional modules evaluated in Section ??; Stage 5 includes the SPPF and C2PSA blocks of YOLOv11.

a channel gate computed from each response weights it:

$$g_d = \sigma(W_2 \delta(W_1 \text{GAP}(E_d))) \in [0, 1]^C, \quad Y = X + \sum_d g_d \odot E_d, \quad (3)$$

where GAP is global average pooling, W_1 and W_2 are 1×1 convolutions with reduction ratio 16, δ is ReLU and σ the sigmoid. The gate is recomputed for every input. Our block tests the dilation-and-gating interpretation of edge selection; the MEIS module of MEIS-YOLO [?] instead uses adaptive pooling, high-pass edge enhancement and a dual-domain selection mechanism, so our results do not transfer to that module directly.

Feature recalibration module (FRM). FRM applies the channel and spatial attention of CBAM [?] to each neck output and blends the result residually, $Y = X + \alpha(X_c - X) + \alpha(X_s - X_c)$, where X_c and X_s are the outputs of the channel and spatial attention steps and $\alpha = 0.5$.

Factorial variants. To separate the ingredients of MEIS, we replaced the dilations (1, 2, 4) with (1, 1, 1), which leaves the parameter count unchanged, and replaced the input-adaptive gate with an input-independent learnable channel weight $\sigma(\theta_d)$, $\theta_d \in \mathbb{R}^C$. The weights θ_d are initialized to zero, so both gate types start close to 0.5. The variant with dilations (1, 1, 1) and a

Table 2: Parameters, computational cost and speed of the evaluated detectors.

Model	Levels	Params (M)	GFLOPs at input size			FPS at input size	
			640	1280	1536	1280	1536
YOLOv11s (P3–P5)	3	9.43	21.6	86.3	124.2	[TBD]	[TBD]
Baseline-P2	4	9.92	37.5	149.8	215.7	[TBD]	[TBD]
Baseline-P2 + MEIS + FRM	4	11.40	44.4	177.4	255.5	[TBD]	[TBD]
dilation (1,1,1), adaptive gate	4	11.40	44.4	177.4	255.5	–	–
dilation (1,2,4), static gate	4	11.25	44.3	177.3	255.3	–	–
dilation (1,1,1), static gate	4	11.25	44.3	177.3	255.3	–	–

GFLOPs are twice the multiply–accumulate operations for one square image with unfused convolution and batch-normalization layers. FPS: one NVIDIA RTX 3080, batch size 1, FP16, network forward pass only.

static gate is a plain residual block of depthwise-separable convolutions with the same capacity as MEIS and no multi-scale or adaptive component.

Cross-scale weighted fusion. This neck, evaluated only in the earlier experiments of Appendix ??, combines resized and channel-projected pyramid features $\tilde{F}_i$ with normalized learnable weights, $F = \sum_i \frac{\text{ReLU}(w_i)}{\sum_j \text{ReLU}(w_j) + \epsilon} \tilde{F}_i$, followed by a depthwise-separable convolution, similar to the fast normalized fusion of BiFPN [?].

Table ?? lists the size and computational cost of each variant.

4.3. Training

Table ?? lists the two schedules. The *progressive* schedule trains for 100, 100, 100 and 50 epochs at 640, 800, 1024 and 1280 px. Each phase restarts a cosine learning-rate schedule with warm-up, and augmentation strength decreases while the box and classification loss weights increase from phase to phase. The *single-phase* schedule trains for all 350 epochs at 1280 px with the phase-1 settings. Both use SGD with momentum 0.937 and weight decay 5×10^{-4} , mixed precision, and the exponential moving average of weights provided by the Ultralytics trainer. Batch sizes were limited by the memory needed for the P2 models at 1280 px and were the same for every model trained with a given schedule.

We report training compute in pixel-epochs, $C = \sum_e S_e^2$, the sum over epochs of the squared input size. Convolutional cost grows with the number of input pixels, so C compares schedules independently of hardware and of other jobs sharing the machine. The progressive schedule costs 2.92×10^8 pixel-epochs and 350 epochs at 1280 px cost 5.73×10^8 .

Table 3: Training schedules. Learning rates follow a cosine curve from lr_0 to $\text{lr}_0 \cdot f$ within each phase.

	Input (px)	Epochs	Batch	lr_0	f	Warm-up (ep.)	Loss weights box / cls / DFL	Mosaic / mixup / copy-paste	Scale / rotation
Progressive, phase 1	640	100	8	0.01	0.01	5	7.5 / 0.5 / 1.5	1.0 / 0.15 / 0.3	0.9 / 10°
Progressive, phase 2	800	100	6	0.005	0.05	3	8.0 / 0.6 / 1.6	0.8 / 0.10 / 0.4	0.8 / 7°
Progressive, phase 3	1024	100	4	0.002	0.08	2	8.5 / 0.7 / 1.7	0.5 / 0.05 / 0.2	0.7 / 5°
Progressive, phase 4	1280	50	2	0.0005	0.10	1	9.0 / 0.8 / 1.8	0 / 0 / 0.1	0.6 / 2°
Single phase	1280	350	2	0.01	0.01	5	7.5 / 0.5 / 1.5	1.0 / 0.15 / 0.3	0.9 / 10°

Mosaic augmentation is switched off for the last 20, 15, 10 and 0 epochs of phases 1–4, and for the last 20 epochs of the single-phase schedule.

Each configuration was trained with seeds 0 and 42. The factorial ablation of Section ?? used a shortened progressive schedule with 25, 20, 20 and 10 epochs per phase and was otherwise identical. Experiments ran on NVIDIA RTX 3080 GPUs (20 GB) with Ultralytics 8.4, PyTorch 2.5.1 and CUDA 12.1.

4.4. Evaluation

All models were evaluated with the Ultralytics validator using letter-box resizing to input size S , a confidence threshold of 0.001, non-maximum suppression at an IoU of 0.5 and at most 600 detections per image. The validation images contain 70.7 annotated instances on average. We report mAP@50 and mAP@50:95 with 101-point interpolation, and precision and recall at the confidence threshold that maximizes the mean F1 score. The primary input size is 1280 px, the final training resolution of both schedules; we also report 1536 px.

4.5. Statistical testing

Differences between two models were tested on the 548 validation images with two paired procedures. Per-image detection statistics were captured from the validator with batch size one, and recomputing mAP from them reproduces the validator’s reported value. In the *paired bootstrap* [?], the images were resampled with replacement 1000 times, both models were scored on each resample, and the 2.5th and 97.5th percentiles of the difference form the 95% confidence interval. The two-sided p -value is twice the smaller fraction of resampled differences lying on either side of zero; when no resample crosses zero we report $p < 0.001$. In the *paired permutation test*, the two models’ statistics were swapped independently for each image with probability 0.5, 1000 times, and $p = (m + 1)/1001$, where m counts permutations whose

Table 4: Progressive and single-phase training of Baseline-P2 on the VisDrone validation set. P and R are precision and recall. Compute is in 10^8 pixel-epochs.

Schedule	Seed	Evaluated at 1280 px				At 1536 px		Compute
		mAP@50	mAP@50:95	P	R	mAP@50	mAP@50:95	
Progressive	0	53.32	31.47	58.9	53.1	54.07	32.01	2.92
	42	53.60	31.51	59.4	52.6	54.26	31.91	2.92
	mean	53.46	31.49	59.2	52.9	54.17	31.96	
Single phase	0	57.26	34.66	62.1	55.1	58.84	35.82	5.73
	42	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	5.73

absolute difference in mAP@50 is at least the observed one. These tests describe uncertainty from the choice of evaluation images for fixed trained models. Variation between training seeds is reported separately, since two seeds are too few for a seed-level interval.

5. Results

5.1. Progressive versus single-phase training

Table ?? compares the two schedules on Baseline-P2. With seed 0, single-phase training was better on every metric at both evaluation sizes: by 3.9 points mAP@50 and 3.2 points mAP@50:95 at 1280px, and by 4.8 and 3.8 points at 1536 px. The paired bootstrap at 1536 px gives a 95% confidence interval of [4.1, 5.5] points ($p < 0.001$), and the permutation test gives $p = 0.001$, the smallest value attainable with 1000 permutations. The two progressive runs differ by 0.28 points at 1280 px, an order of magnitude less than the gap between schedules. The second single-phase seed is **[pending]**.

The single-phase run also needed less compute to reach the progressive result. Its checkpoint after 61 epochs, which corresponds to 34.3% of the progressive schedule’s compute, scored 53.34% mAP@50 and 31.66% mAP@50:95 at 1280 px, against 53.32% and 31.47% for the finished progressive model (Fig. ??a). After 181 epochs, at about the same compute as the whole progressive schedule (101.6%), it reached 57.19%, 3.87 points ahead. That checkpoint comes from the middle of a 350-epoch run whose learning rate had not yet annealed, so a run planned to stop there would probably score at least as high. Later epochs added little: the best checkpoint of the full run, at epoch 247, scored 57.26%, only 0.07 points more.

All ten classes improved (Fig. ??). The largest gain was for bicycles (+7.4 AP@50 at 1280 px), followed by vans and tricycles (+4.7 each); the smallest

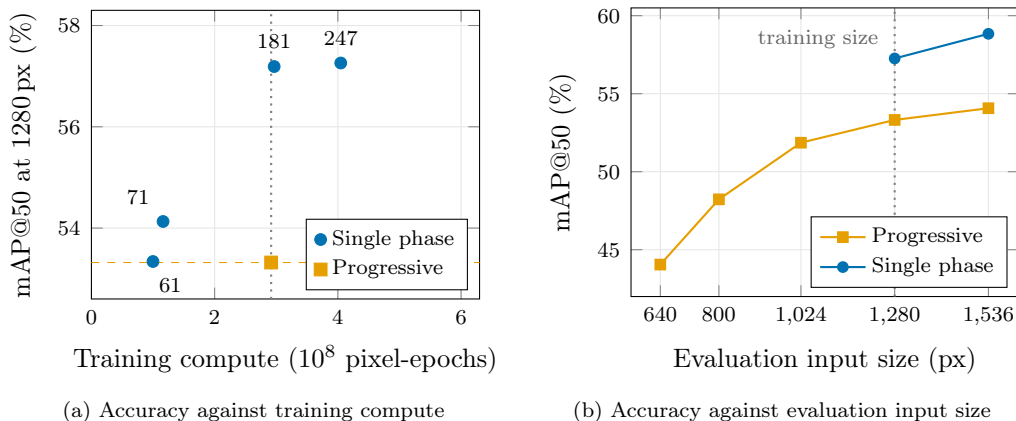


Figure 4: (a) mAP@50 of single-phase checkpoints after 61, 71 and 181 epochs and of the best checkpoint (epoch 247), compared with the finished progressive model (dashed line). The dotted line marks the progressive schedule’s compute. (b) The same seed-0 models evaluated at different input sizes.

were for cars (+0.8), which were already above 89 AP, and awning-tricycles (+1.3). Bicycles are among the thinnest objects in the dataset, which fits prediction P1, but large vehicles such as buses and trucks also gained more than four points, so per-class results alone do not show that the gain is concentrated on small objects. Section ?? tests P1 by object size directly.

Both models kept improving as the evaluation size grew (Fig. ??b). The progressive model rose from 44.1% at 640 px to 54.1% at 1536 px, including a gain of 0.8 points from 1280 to 1536 px, above its final training size. The single-phase model gained 1.6 points over the same step. This is consistent with the sampling argument and with the train–test resolution effect described by Touvron et al. [?], although neither explanation is tested directly by these two points.

5.2. Detection stride

Table ?? compares YOLOv11s with three detection levels and Baseline-P2, both trained in a single phase at 1280 px with seed 0. **[Describe the result once the P3–P5 run is evaluated.]** Adding the P2 level costs 0.49M parameters and raises GFLOPs by 74% at every input size (Table ??), because the stride-4 level operates on a feature map with four times as many positions as the stride-8 level.

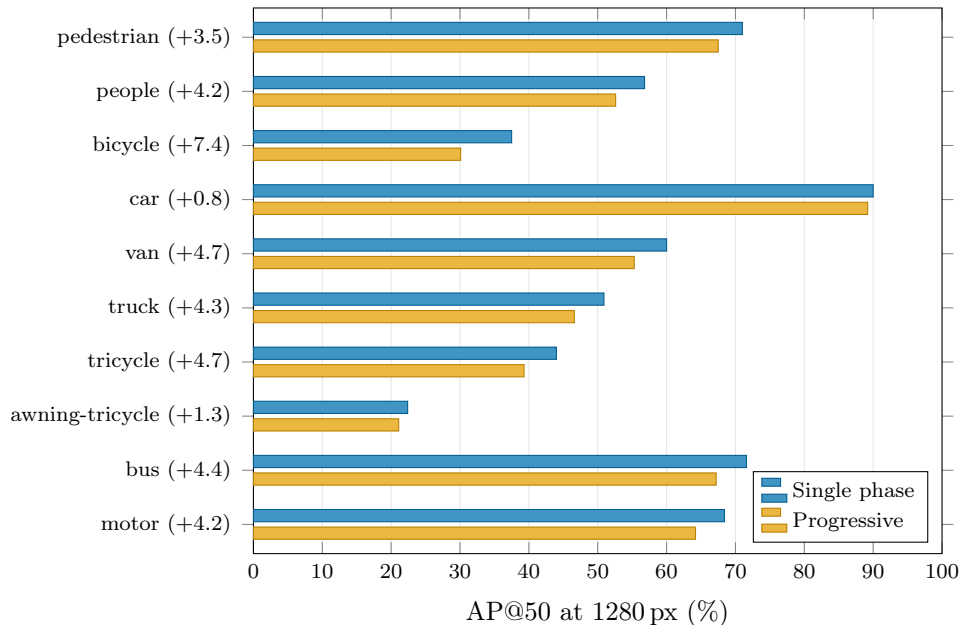


Figure 5: Per-class AP@50 of Baseline-P2 at 1280px (seed 0). Numbers in parentheses give the gain from single-phase training.

5.3. Feature modules

Table ?? adds MEIS and FRM to Baseline-P2 under the full progressive schedule. The change in mAP@50 was +0.56 points with seed 0 and -0.36 points with seed 42 at 1280px, and +0.58 and -0.34 points at 1536px. Neither difference is significant (Fig. ??): the paired bootstrap intervals at 1536px are $[-0.15, 1.27]$ and $[-1.00, 0.33]$, with $p = 0.11$ and $p = 0.41$, and the permutation tests agree ($p = 0.12$ and $p = 0.37$). The mAP@50:95 differences, +0.17 and -0.13 points, are smaller still ($p = 0.48$ and $p = 0.65$). The same test found the schedule effect of Section ?? with $p < 0.001$, so the null result does not come from an insensitive test. Even on the favourable seed, the interval excludes improvements larger than about 1.3 points. For this, the modules cost 1.48M parameters and 18% more GFLOPs (Table ??).

5.3.1. Factorial ablation of MEIS

Table ?? and Fig. ?? separate the two ingredients of MEIS under the shortened schedule. All four MEIS variants include FRM, and the reference model has neither module. The model without modules was the most accurate, and removing either ingredient of MEIS made results slightly better

Table 5: Effect of the stride-4 detection level. Both models were trained in a single phase at 1280 px (seed 0).

Model	1280 px		1536 px	
	mAP@50	mAP@50:95	mAP@50	mAP@50:95
YOLOv11s (P3–P5)	[TBD]	[TBD]	[TBD]	[TBD]
Baseline-P2	57.26	34.66	58.84	35.82
Δ	[TBD]	[TBD]	[TBD]	[TBD]

Table 6: Adding MEIS and FRM to Baseline-P2 under the full progressive schedule. The paired tests are computed at 1536 px over the 548 validation images.

Seed	Model	1280 px		1536 px		Paired test, mAP@50	
		mAP@50	mAP@50:95	mAP@50	mAP@50:95	95% CI	p (bootstrap / perm.)
0	Baseline-P2	53.32	31.47	54.07	32.01	[−0.15, +1.27]	0.11 / 0.12
	+ MEIS + FRM	53.88	31.96	54.65	32.18		
	Δ	+0.56	+0.49	+0.58	+0.17		
42	Baseline-P2	53.60	31.51	54.26	31.91	[−1.00, +0.33]	0.41 / 0.37
	+ MEIS + FRM	53.24	31.21	53.92	31.78		
	Δ	−0.36	−0.30	−0.34	−0.13		

The significance tests re-validate both models with batch size one, which changes mAP@50 by at most 0.01 points relative to the table.

rather than worse. Averaged over paired same-seed differences at 1280 px, multi-scale dilation changed mAP@50 by -0.20 points and input-adaptive gating by -0.24 points, each negative in all four pairings, with a negligible interaction (-0.06). Full MEIS scored 0.44 points below the plain convolution of equal size. At 1536 px the effects pointed the same way (-0.16 and -0.18 points), in three of four pairings each. With two seeds, differences of 0.2 points cannot be resolved statistically; what the ablation does show is that neither ingredient helped.

The module variants trailed the no-module model by 0.6 to 1.0 points in this 75-epoch schedule, whereas under the full schedule the gap was not significant (Table ??). A plausible reading is that the added blocks slow optimization early on and, given enough epochs, settle close to an identity mapping through their residual connections, but we have not tested this directly.

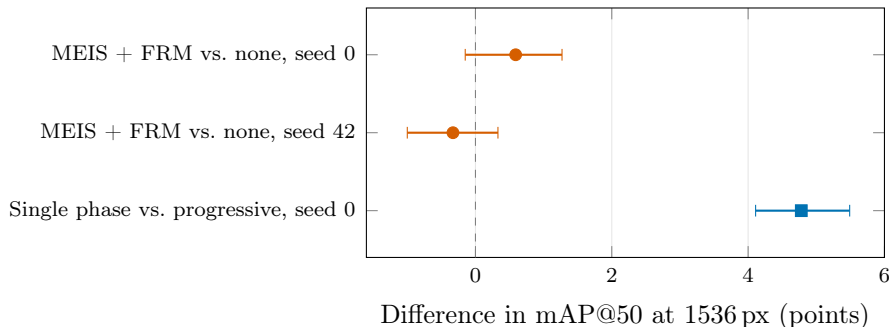


Figure 6: Paired differences with 95% bootstrap confidence intervals over the 548 validation images. The module intervals include zero; the schedule interval does not.

Table 7: Factorial ablation of MEIS on Baseline-P2 with the 75-epoch progressive schedule (mean $\pm$ s.d. over seeds 0 and 42).

Dilations	Gate	Params (M)	1280 px		1536 px	
			mAP@50	mAP@50:95	mAP@50	mAP@50:95
No module		9.92	51.30 $\pm$ 0.10	29.71	51.92 $\pm$ 0.18	30.03
(1,1,1)	static	11.25	50.71 $\pm$ 0.05	29.37	51.18 $\pm$ 0.40	29.79
(1,2,4)	static	11.25	50.54 $\pm$ 0.16	29.15	51.14 $\pm$ 0.16	29.52
(1,1,1)	adaptive	11.40	50.50 $\pm$ 0.11	29.29	51.12 $\pm$ 0.62	29.57
(1,2,4)	adaptive (MEIS)	11.40	50.27 $\pm$ 0.13	29.08	50.84 $\pm$ 0.47	29.26
Dilation (1,2,4) vs. (1,1,1)			−0.20		−0.16	
Adaptive vs. static gate			−0.24		−0.18	
MEIS vs. (1,1,1) with static gate			−0.44		−0.34	

All MEIS variants also contain FRM. Effects in the lower block are means of paired same-seed differences in mAP@50.

5.4. Seed variation and image sampling

The two sources of uncertainty were of different size. For the MEIS and FRM comparison, the paired difference moved from +0.58 to -0.34 points at 1536 px between seeds, a standard deviation of 0.65 points, while the image-level bootstrap standard error was 0.35 points. Individual models also moved: Baseline-P2 scored 53.32% and 53.60% at 1280 px with the two seeds, and the MEIS model 53.88% and 53.24%. Two seeds give only a rough estimate of the seed variance, but the comparison is enough to show that a single seed would have been misleading here. Seed 0 alone suggests that the modules add about half a point; seed 42 suggests that they remove a third of one. For effects below one point we would recommend at least three seeds together with paired tests over the evaluation images.

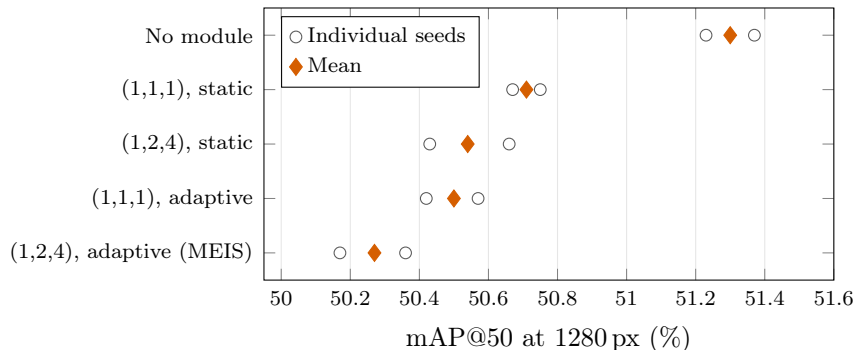


Figure 7: Factorial ablation of MEIS (Table ??). Labels give the branch dilations and the gate type.

Table 8: AP@50 at 1280 px by object area, using COCO area ranges in original image pixels (seed 0).

Model and schedule	Small ($< 32^2$)	Medium	Large ($> 96^2$)
Baseline-P2, progressive	[TBD]	[TBD]	[TBD]
Baseline-P2, single phase	[TBD]	[TBD]	[TBD]
YOLOv11s (P3–P5), single phase	[TBD]	[TBD]	[TBD]

5.5. Accuracy by object size

Table ?? splits AP@50 by object area using the COCO ranges [?], computed in original image pixels. **[Describe once the size-stratified evaluation is available, and state whether the gains concentrate on small objects as P1 predicts.]**

5.6. Cross-dataset evaluation on UAVDT

We applied the VisDrone-trained models to UAVDT [?] without further training. UAVDT annotates three vehicle classes, which we mapped to the VisDrone classes car, truck and bus, and Table ?? lists the results. **[Describe the frame sampling, the handling of UAVDT ignore regions and of VisDrone “van” predictions, and the results.]** Because UAVDT contains no pedestrians, bicycles or tricycles, this evaluation says nothing about small non-vehicle objects outside VisDrone.

Table 9: Zero-shot AP@50 on UAVDT.

Model and schedule	Car	Truck	Bus	mAP@50
Baseline-P2, progressive	[TBD]	[TBD]	[TBD]	[TBD]
Baseline-P2, single phase	[TBD]	[TBD]	[TBD]	[TBD]

Table 10: Reported VisDrone-DET2019 results with their evaluation protocols. The rows are not directly comparable.

Method	Split	Input (px)	COCO initialization	AP@50	Params (M)
YOLOv11m, as reported in [?]	test	640	no	34.6	20.04
MEIS-YOLO [?]	test	640	no	36.0	15.04
Baseline-P2, progressive (ours)	val	1280	yes	53.3	9.92
Baseline-P2, single phase (ours)	val	1280	yes	57.3	9.92

5.7. Relation to published results

Published VisDrone results are difficult to compare with each other because evaluation split, input size, pretraining and inference procedure differ between papers. Table ?? shows the problem with the closest related method. MEIS-YOLO [?] reports 36.0% AP@50 on the VisDrone-DET2019 test set, for a network trained from scratch at 640 px, against 34.6% for its YOLOv11m baseline. Our models were evaluated on the validation set, start from COCO weights and use 1280 px inputs, so the numbers are not on a common scale and the table should not be read as a ranking. Region-based detectors that process magnified crops [? ? ?] use several network passes per image and belong to a different accuracy–cost regime.

6. Discussion

6.1. Why did the progressive schedule do worse?

The two schedules differ in more than resolution. The progressive schedule spends 200 of its 350 epochs at 640 or 800 px, where Table ?? predicts that objects smaller than 10 to 12.5 pixels cannot be assigned targets even at stride 4. It also restarts the learning rate with a warm-up at every phase boundary, which raises the learning rate again after the previous phase has annealed, and it changes augmentation and loss weights between phases. We did not separate these factors, so we cannot say how much each contributed; testing a single-resolution schedule with phase-wise restarts, or a progressive

schedule without them, would answer that. The checkpoint results do rule out one argument for the progressive schedule, namely faster convergence, because single-phase training matched its final accuracy after 61 epochs.

6.2. *Why did the modules not help?*

MEIS and FRM act on features that have already been sampled at stride r . By the argument of Section ??, they can reweight what the backbone recorded but cannot recover objects that fell between anchor points, and their residual design lets them stay close to an identity mapping when reweighting does not help. The factorial ablation fits this reading: the variants differed from each other by a few tenths of a point, regardless of dilation or gating. Two caveats apply. Our MEIS block is not the MEIS module of MEIS-YOLO, and the module experiments used the progressive schedule; whether they behave differently when trained at 1280 px throughout remains untested.

6.3. *Practical recommendations*

Three recommendations follow from these experiments. First, train at the input size that will be used at deployment before trying resolution schedules. Second, before adding modules, check the finest detection stride against the object size distribution with Eq. (??). Third, compare designs with at least two or three seeds, test paired differences over the evaluation images, and report training compute alongside accuracy.

6.4. *Limitations*

The study has several limitations. Most configurations were trained with two seeds, and the single-phase schedule **[with one seed / with two seeds]**. VisDrone is the only dataset with small non-vehicle objects in our evaluation, and UAVDT covers only vehicles. We studied one detector family at one size, so larger models and transformer-based detectors may behave differently; we chose YOLOv11s because it is widely used, has a maintained implementation with pretrained weights, and is a realistic size for drone platforms. The schedule comparison changes resolution, learning-rate restarts and augmentation together. The module experiments used the progressive schedule, and the factorial ablation used a shortened one. We did not evaluate robustness to motion blur, noise or illumination changes. Finally, the sampling argument is a heuristic, and FPS figures depend on the hardware.

7. Conclusion

For YOLOv11s on VisDrone-DET2019, the design decisions that changed small-object accuracy were the ones that changed how densely objects are sampled. Training at 1280 px in a single phase outperformed a four-phase progressive schedule by 3.9 points mAP@50 at the same epoch budget and matched its accuracy with a third of the compute, and the stride-4 detection level **[result]**. Edge selection with adaptive gating and CBAM-style recalibration did not produce measurable gains once runs were matched and repeated, and neither multi-scale dilation nor adaptive gating did better than a plain convolution of equal size. These results do not show that feature modules cannot help small-object detection, but they suggest that such modules should be compared against baselines trained at the same resolution and budget, with repeated seeds and paired tests, before their gains are attributed to the modules themselves.

CRediT authorship contribution statement

[Complete with the CRediT roles of each author.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. **[Confirm.]**

Data availability

VisDrone-DET2019 and UAVDT are publicly available. **[State whether and where the code, configurations and trained weights will be released.]**

Declaration of generative AI and AI-assisted technologies in the writing process

[Complete according to the journal’s policy, describing any AI tools used in preparing the manuscript and the code.]

Table A.11: Earlier ablations with three detection levels, the full progressive schedule and evaluation at 1024px. One seed per row unless stated.

Configuration	mAP@50	mAP@50:95
PANet neck, no module	49.84	29.61
PANet neck + MEIS	48.15	27.96
Weighted fusion (three levels at P4) + MEIS	47.91	27.96
+ FRM after fusion, $\alpha = 0.5$ (three runs)	48.50 ± 0.28	28.47 ± 0.27
+ FRM before fusion, $\alpha = 0.5$	48.02	28.15
+ FRM after fusion, $\alpha = 0.3$	47.90	28.13
+ FRM after fusion, $\alpha = 0.7$	48.91	28.62
Weighted fusion (two levels at P4) + MEIS + FRM	49.85	29.56

Acknowledgements

The authors thank the VisDrone and UAVDT teams for the benchmarks and the Ultralytics team for the YOLOv11 implementation.

Appendix A. Earlier single-seed ablations

Before the controlled experiments above, we ran a set of single-seed ablations with an earlier configuration. It differed from the main experiments in four ways: the detectors used three detection levels (P3 to P5), module variants did not receive copied head weights and so started with randomly initialized necks while the reference neck was pretrained, models were evaluated at 1024px, and at most 300 detections were kept per image. Table ?? reports these runs for completeness, including the placement of FRM before or after fusion, the residual blend factor α , and two- versus three-level fusion at P4.

No module configuration exceeded the plain PANet neck. Three repeated runs of the same configuration spanned 48.25% to 48.80% mAP@50, and most other differences among module variants lie within about twice that spread. Two-level fusion scored 1.35 points above the mean of the three-level runs, the opposite of what simultaneous three-level fusion was designed to achieve, and equalled the no-module neck. Because the module variants in this configuration started from randomly initialized necks and each was run once, we do not draw conclusions from these numbers beyond the absence of a gain over the PANet neck.